

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name			
		Mr. S Naresh Kumar	
		Ms. B. Swathi	
		Dr. Sasanko Shekhar Gantayat	
		Mr. Md Sallauddin	
		Dr. Mathivanan	
		Mr. Y Srikanth	
		Ms. N Shilpa	
		Dr. Rishabh Mittal (Coordinator)	
		Dr. R. Prashant Kumar	
		Mr. Ankushavali MD	
		Mr. B Viswanath	
		Ms. Sujitha Reddy	
		Ms. A. Anitha	
		Ms. M.Madhuri	
		Ms. Katherashala Swetha	
		Ms. Velpula sumalatha	
Mr. Bingi Raju			
CourseCode	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/II	Regulation	R23
Date and Day of Assignment	Week6 – Wednesday	Time(s)	23CSBTB01 To 23CSBTB52
Name:	M.AnuChandra	Holl.no	2303A54070
Assignment Number:12.3(Present assignment number)/24(Total number of assignments)			
Q.No.	Question	Expected Time to complete	
1	Lab 12: Algorithms with AI Assistance Sorting, Searching, and Algorithm Optimization Using AI Tools Lab Objectives	Week3 - Wednesday	

- The objectives of this laboratory exercise are to:
- Apply AI-assisted programming techniques to implement sorting and searching algorithms.
 - Analyze and compare algorithm efficiency using time and space complexity.
 - Understand how AI tools can suggest optimizations and alternative algorithmic approaches.
 - Strengthen problem-solving skills through real-world, data-driven scenarios.

Learning Outcomes

- After completing this lab, students will be able to:
- Implement and optimize classic algorithms using AI-assisted coding tools.
 - Compare multiple algorithms for the same problem and justify their selection.
 - Measure and analyze runtime performance using experimental data.
 - Critically review and refine AI-generated algorithmic solutions.

Task 1: Sorting Student Records for Placement Drive Scenario

SR University’s Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by **CGPA in descending order**.

Tasks

1. Use **GitHub Copilot** to generate a program that stores student records (Name, Roll Number, CGPA).
2. Implement the following sorting algorithms using AI assistance:
 - Quick Sort
 - Merge Sort
3. Measure and compare runtime performance for large datasets.
4. Write a function to display the **top 10 students** based on CGPA.

Expected Outcome

- Correctly sorted student records.
- Performance comparison between Quick Sort and Merge Sort.
- Clear output of top-performing students.

```
#!/usr/bin/env python3
# SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.
# Write a program to generate a program that stores student records.
# (Name, Roll Number, CGPA).
# 1. Implement the following sorting algorithms using AI assistance.
# a. Quick Sort
# b. Merge Sort
# 2. Measure and compare runtime performance for large datasets.
# 3. Write a function to display the top 10 students based on CGPA. Give an example also.
import sys
import time

class Student:
    def __init__(self, name, roll, cgpa):
        self.name = name
        self.roll = roll
        self.cgpa = cgpa

    def __repr__(self):
        return f"({self.name} | Roll: {self.roll} | CGPA: {self.cgpa})"

def quick_sort(students):
    if len(students) <= 1:
        return students

    pivot = students[len(students)//2].cgpa

    left = [s for s in students if s.cgpa > pivot]
    middle = [s for s in students if s.cgpa == pivot]
    right = [s for s in students if s.cgpa < pivot]

    return quick_sort(left) + middle + quick_sort(right)

def merge_sort(students):
    if len(students) <= 1:
        return students

    mid = len(students)//2
    left = merge_sort(students[:mid])
    right = merge_sort(students[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i].cgpa >= right[j].cgpa:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result

# Example usage:
if __name__ == "__main__":
    # Sample student records
    students = [
        Student("Alice", 101, 8.5),
        Student("Bob", 102, 7.8),
        Student("Charlie", 103, 9.2),
        Student("David", 104, 6.5),
        Student("Eve", 105, 8.9),
        Student("Frank", 106, 7.1),
        Student("Grace", 107, 8.3),
        Student("Heidi", 108, 7.6),
        Student("Ivan", 109, 8.7),
        Student("Julia", 110, 7.4),
        Student("Kevin", 111, 8.1),
        Student("Liam", 112, 7.9),
        Student("Mia", 113, 8.6),
        Student("Noah", 114, 7.3),
        Student("Olivia", 115, 8.4),
        Student("Peter", 116, 7.7),
        Student("Quinn", 117, 8.0),
        Student("Ryan", 118, 7.5),
        Student("Sophia", 119, 8.2),
        Student("Theodore", 120, 7.0)
    ]

    # Sort using Quick Sort
    start_time = time.time()
    sorted_students_qs = quick_sort(students)
    end_time = time.time()
    time_qs = end_time - start_time

    # Sort using Merge Sort
    start_time = time.time()
    sorted_students_ms = merge_sort(students)
    end_time = time.time()
    time_ms = end_time - start_time

    # Display top 10 students based on CGPA
    top_10_students = sorted_students_ms[:10]

    print("\nTop 10 Students based on CGPA:")
    for student in top_10_students:
        print(student)

    print(f"\nQuick Sort Time: {time_qs} seconds")
    print(f"Merge Sort Time: {time_ms} seconds")
```

```
ai_ass.12.3 > task01.py > ...
42 def merge(left, right):
43     if not left:
44         return right
45     if not right:
46         return left
47     else:
48         result.append(right[j])
49         j += 1
50
51     result.extend(left[i:])
52     result.extend(right[j:])
53     return result
54
55 def display_top_10(students):
56     print("\n 🏆 Top 10 Students (by CGPA):")
57     for i, student in enumerate(students[:10], start=1):
58         print(f"{i}. {student}")
59
60 def generate_students(n):
61     students = []
62     for i in range(n):
63         name = f"Student{i+1}"
64         roll = 1000 + i
65         cgpa = round(random.uniform(5.0, 10.0), 2)
66         students.append(Student(name, roll, cgpa))
67     return students
68
69 if __name__ == "__main__":
70
71     # Example dataset size
72     N = 5000
73
74     students = generate_students(N)
75
76     # ----- Quick Sort Timing -----
77     start = time.time()
78     quick_sorted = quick_sort(students)
79     quick_time = time.time() - start
80
81     # ----- Merge Sort Timing -----
82     start = time.time()
83     merge_sorted = merge_sort(students)
84     merge_time = time.time() - start
85
86     # Runtime comparison
87     print("\n ⚡ Runtime Comparison")
88     print(f"Quick Sort Time : {quick_time:.5f} seconds")
89     print(f"Merge Sort Time : {merge_time:.5f} seconds")
90
91     # Display Top Students
92     display_top_10(quick_sorted)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding> & C:\Users\MYPC
4. Student384 | Roll: 1383 | CGPA: 9.99
5. Student605 | Roll: 1604 | CGPA: 9.99
6. Student1177 | Roll: 2176 | CGPA: 9.99
7. Student1347 | Roll: 2346 | CGPA: 9.99
8. Student1372 | Roll: 2371 | CGPA: 9.99
9. Student1702 | Roll: 2701 | CGPA: 9.99
10. Student1937 | Roll: 2936 | CGPA: 9.99
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding> █
```

Task 2: Implementing Bubble Sort with AI Comments

- **Task:** Write a Python implementation of **Bubble Sort**.
- **Instructions:**
 - Students implement Bubble Sort normally.
 - Ask AI to generate **inline comments explaining key logic** (like swapping, passes, and termination).
 - Request AI to provide **time complexity analysis**.
- **Expected Output:**
 - A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis.

```

ai_ass.123 > task02.py > _
1 # Implementing Bubble Sort with AI Comments
2 # * Task: Write a Python implementation of Bubble Sort.
3 # * Instructions:
4 # * Students implement Bubble Sort normally.
5 # * Ask AI to generate inline comments explaining key logic (like
6 # swapping, passes, and termination).
7 # * Request AI to provide time complexity analysis.
8
9 def bubble_sort(arr):
10     n = len(arr)
11
12     for i in range(n):
13         swapped = False
14
15         for j in range(0, n - i - 1):
16             if arr[j] > arr[j + 1]:
17                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
18                 swapped = True
19
20         if not swapped:
21             break
22
23     return arr
24
25
26 numbers = [64, 34, 25, 12, 22, 11, 90]
27
28 print("Original List:", numbers)
29
30 sorted_numbers = bubble_sort(numbers)
31
32 print("Sorted List:", sorted_numbers)

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS POSTMAN CONSOLE

```

PS C:\Users\WYPC\OneDrive\Desktop\ai assisted coding & C:\Users\WYPC\AppData\Local\Programs\Python\Python39\Scripts>
7. Student1347 | Roll: 2346 | CGPA: 9.99
8. Student1372 | Roll: 2371 | CGPA: 9.99
9. Student1702 | Roll: 2701 | CGPA: 9.99
10. Student1937 | Roll: 2936 | CGPA: 9.99
PS C:\Users\WYPC\OneDrive\Desktop\ai assisted coding & C:\Users\WYPC\AppData\Local\Programs\Python\Python39\Scripts>
Original List: [64, 34, 25, 12, 22, 11, 90]
Sorted List: [11, 12, 22, 25, 34, 64, 90]
PS C:\Users\WYPC\OneDrive\Desktop\ai assisted coding & C:\Users\WYPC\AppData\Local\Programs\Python\Python39\Scripts>

```

Task 3: Quick Sort and Merge Sort Comparison

- **Task:** Implement Quick Sort and Merge Sort using recursion.
- **Instructions:**
 - Provide AI with partially completed functions for recursion.
 - Ask AI to complete the missing logic and add docstrings.
 - Compare both algorithms on random, sorted, and reverse-sorted lists.
- **Expected Output:**
 - Working Quick Sort and Merge Sort implementations.
 - AI-generated explanation of average, best, and worst-case complexities.

```
ai_ass.12.3 > task03.py > ...
1 # Quick Sort and Merge Sort Comparison
2 # * Task: Implement Quick Sort and Merge Sort using recursion.
3 # * Instructions:
4 # * Provide AI with partially completed functions for recursion.
5 # * Ask AI to complete the missing logic and add docstrings.
6 # * Compare both algorithms on random, sorted, and reverse-sorted
7 # lists.
8
9 import random
10 import time
11
12
13 def quick_sort(arr):
14     if len(arr) <= 1:
15         return arr
16
17     pivot = arr[len(arr) // 2]
18
19     left = [x for x in arr if x < pivot]
20     middle = [x for x in arr if x == pivot]
21     right = [x for x in arr if x > pivot]
22
23     return quick_sort(left) + middle + quick_sort(right)
24
25
26 def merge_sort(arr):
27     if len(arr) <= 1:
28         return arr
29
30     mid = len(arr) // 2
31
32     left_half = merge_sort(arr[:mid])
33     right_half = merge_sort(arr[mid:])
34
35     return merge(left_half, right_half)
36
37
38 def merge(left, right):
39     result = []
40     i = j = 0
41
42     while i < len(left) and j < len(right):
43         if left[i] <= right[j]:
44             result.append(left[i])
45             i += 1
46         else:
47             result.append(right[j])
48             j += 1
49
```

```

ai_ass.12.3 > task.03.py > ...
38 def merge(left, right):
49     result.extend(left[i:])
51     result.extend(right[j:])
52     return result
53
54
55 def test_algorithm(sort_function, data):
56     start = time.time()
57     sort_function(data.copy())
58     return time.time() - start
59
60
61 size = 5000
62
63 random_list = [random.randint(1, 10000) for _ in range(size)]
64 sorted_list = sorted(random_list)
65 reverse_sorted_list = sorted(random_list, reverse=True)
66
67 print("\n--- Performance Comparison ---")
68
69 datasets = {
70     "Random List": random_list,
71     "Sorted List": sorted_list,
72     "Reverse Sorted List": reverse_sorted_list
73 }
74
75 for name, dataset in datasets.items():
76     quick_time = test_algorithm(quick_sort, dataset)
77     merge_time = test_algorithm(merge_sort, dataset)
78
79     print(f"\n{name}")
80     print(f"Quick Sort Time : {quick_time:.5f} seconds")
81     print(f"Merge Sort Time : {merge_time:.5f} seconds")

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```

PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding> & C:\Users\MYPC\AppData
Sorted List
Quick Sort Time : 0.00479 seconds
Merge Sort Time : 0.00438 seconds

Reverse Sorted List
Quick Sort Time : 0.00379 seconds
Merge Sort Time : 0.00477 seconds

```

Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store’s inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

Task:

- Use AI to suggest the most efficient search and sort algorithms for this use case.
- Implement the recommended algorithms in Python.
- Justify the choice based on dataset size, update frequency, and performance requirements.

Expected Output:

- A table mapping operation → recommended algorithm → justification.
- Working Python functions for searching and sorting the inventory.

```

# Real-Time Application - Inventory Management System
# Scenario: A retail store's inventory system contains thousands of products,
# each with attributes like product ID, name, price, and stock quantity. Store staff
# need to:
# 1. Quickly search for a product by ID or name.
# 2. Sort products by price or quantity for stock analysis.
# Task:
# • Use AI to suggest the most efficient search and sort algorithms for this
# use case.
# • Implement the recommended algorithms in Python.
# • Justify the choice based on dataset size, update frequency, and
# performance requirements.
class Product:
    def __init__(self, pid, name, price, quantity):
        self.pid = pid
        self.name = name
        self.price = price
        self.quantity = quantity

    def __repr__(self):
        return f"{self.pid} | {self.name} | Price:{self.price} | Qty:{self.quantity}"

class Inventory:
    def __init__(self):
        self.products = []
        self.id_index = {}
        self.name_index = {}

    def add_product(self, product):
        self.products.append(product)
        self.id_index[product.pid] = product
        self.name_index[product.name.lower()] = product

    def search_by_id(self, pid):
        return self.id_index.get(pid, "Product not found")

    def search_by_name(self, name):
        return self.name_index.get(name.lower(), "Product not found")

    def sort_by_price(self):
        return sorted(self.products, key=lambda x: x.price)

    def sort_by_quantity(self):
        return sorted(self.products, key=lambda x: x.quantity)

inventory = Inventory()

```

```

ai_ass.12.3 > task.04.py > ...
47
48     inventory = Inventory()
49
50     inventory.add_product(Product(101, "Laptop", 75000, 15))
51     inventory.add_product(Product(102, "Mouse", 500, 120))
52     inventory.add_product(Product(103, "Keyboard", 1500, 60))
53     inventory.add_product(Product(104, "Monitor", 12000, 25))
54     inventory.add_product(Product(105, "Headphones", 2500, 40))
55
56     print("\nSearch by ID:")
57     print(inventory.search_by_id(103))
58
59     print("\nSearch by Name:")
60     print(inventory.search_by_name("mouse"))
61
62     print("\nSorted by Price:")
63     for p in inventory.sort_by_price():
64         print(p)
65
66     print("\nSorted by Quantity:")
67     for p in inventory.sort_by_quantity():
68         print(p)

```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE
PS C:\Users\WYPC\OneDrive\Desktop\ai assisted coding> & C:\Users\WYPC\
Sorted by Quantity:
101 | Laptop | Price:75000 | Qty:15
104 | Monitor | Price:12000 | Qty:25
105 | Headphones | Price:2500 | Qty:40
103 | Keyboard | Price:1500 | Qty:60
102 | Mouse | Price:500 | Qty:120
PS C:\Users\WYPC\OneDrive\Desktop\ai assisted coding>

```

Task 5: Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered **FinTech Lab** at SR University is building a tool for analyzing **stock price movements**. The requirement is to quickly **sort stocks by daily gain/loss** and search for specific stock symbols efficiently.

- Use **GitHub Copilot** to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).
- Implement sorting algorithms to rank stocks by **percentage change**.
- Implement a **search function** that retrieves stock data instantly when a stock symbol is entered.
- Optimize sorting with **Heap Sort** and searching with **Hash Maps**.
- Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

```
#main123.py > task01.py >...
1 # Final Year Stock Data Sorting & Searching
2 # Scenario:
3 # An AI-powered Fintech Lab at SR University is building a tool for analyzing
4 # stock price movements. The requirement is to quickly sort stocks by daily
5 # gain/loss and search for specific stock symbols efficiently.
6 # Use GitHub Copilot to fetch or simulate stock price data (Stock
7 # Symbol, Opening Price, Closing Price).
8 # Implement sorting algorithms to rank stocks by percentage change.
9 # Implement a search function that retrieves stock data instantly when a
10 # stock symbol is entered.
11 # Optimize sorting with Heap Sort and searching with Hash Maps.
12 # Compare performance with standard library functions (sorted(), dict
13 # lookups) and analyze trade-offs.
14 # Note: Report should be submitted as a word document for all tasks in a single
15 # document with prompts, comments & code explanation, and output and if
16 # required, screenshots.
17
18 import random
19 import time
20
21
22 class Stock:
23     def __init__(self, symbol, open_price, close_price):
24         self.symbol = symbol
25         self.open_price = open_price
26         self.close_price = close_price
27         self.change = ((close_price - open_price) / open_price) * 100
28
29     def __repr__(self):
30         return f"[{self.symbol} | Open:{self.open_price:.2f} | Close:{self.close_price:.2f} | Change:{self.change:.2f}]"
31
32
33 def heapify(arr, n, i):
34     largest = i
35     left = 2 * i + 1
36     right = 2 * i + 2
37
38     if left < n and arr[left].change > arr[largest].change:
39         largest = left
40
41     if right < n and arr[right].change > arr[largest].change:
42         largest = right
43
44     if largest != i:
45         arr[i], arr[largest] = arr[largest], arr[i]
46         heapify(arr, n, largest)
47
48
```



```

ai_ass.12.3 > task05.py > ...
49 def heap_sort(arr):
50     for i in range(n // 2 - 1, -1, -1):
51         heapify(arr, n, i)
52
53     for i in range(n - 1, 0, -1):
54         arr[i], arr[0] = arr[0], arr[i]
55         heapify(arr, i, 0)
56
57     return arr[::-1]
58
59
60
61
62 def generate_stocks(n):
63     stocks = []
64     for i in range(n):
65         symbol = f"STK{i}"
66         open_price = random.uniform(100, 1000)
67         close_price = open_price + random.uniform(-50, 50)
68         stocks.append(Stock(symbol, open_price, close_price))
69     return stocks
70
71
72 def build_hash_map(stocks):
73     return {stock.symbol: stock for stock in stocks}
74
75
76 def search_stock(stock_map, symbol):
77     return stock_map.get(symbol, "Stock not found")
78
79
80 stocks = generate_stocks(1000)
81
82 start = time.time()
83 heap_sorted = heap_sort(stocks.copy())
84 heap_time = time.time() - start
85
86 start = time.time()
87 builtin_sorted = sorted(stocks, key=lambda x: x.change, reverse=True)
88 builtin_time = time.time() - start
89
90 stock_map = build_hash_map(stocks)
91
92 print("Heap Sort Time:", heap_time)
93 print("Built-in Sort Time:", builtin_time)
94
95 print("\nSearch Result:")
96 print(search_stock(stock_map, "STK10"))
97
98 print("\nTop 5 Stocks by Percentage Change:")
99 for stock in heap_sorted[:5]:
100     print(stock)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding> & C:\Users\MYPC\AppData\Local

Top 5 Stocks by Percentage Change:

STK386		Open:101.12		Close:150.37		Change:48.70%
STK667		Open:102.67		Close:150.80		Change:46.88%
STK73		Open:107.07		Close:154.30		Change:44.11%
STK722		Open:109.05		Close:155.73		Change:42.81%
STK555		Open:111.89		Close:158.20		Change:41.39%

PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding>